AI Compiler Performance Engineering

Auto-Loops Books — ch14

Contents

1 MLIR: Modern Compiler Infrastructure **1**

1.1 MLIR HW matrix 2
1.2 MLIR dialect stack for decode export 4
1.3 MLIR arch passes 5
1.4 MLIR codegen 7
1.5 MLIR tuning 8
1.6 MLIR case study 9
1.7 YiRage MLIR integration notes 12
1.8 MLIR decode review gates 13
1.9 MLIR decode benchmarking methodology 17
1.10 Key Takeaways 23
1.11 Conclusion 24

Bibliography **25**

Chapter 1

MLIR: Modern Compiler Infrastructure

The slide deck says "one IR, every target." The production dashboard says something else: the same PyTorch-exported decoder graph, lowered through MLIR, can differ by an order of magnitude in p99 latency across GPU, CPU, and NPU fleets (Davies *et al.*, 2025; Chen, 2020). That gap is rarely a missing dialect—it is a missing *contract*. Chapter 3 mapped residency and legality per hardware class; this chapter shows how MLIR must encode those predicates in IR attributes, pass gates, and emit paths before any canonicalizer runs (AMD, 2024b; Chen and YiRage Contributors, 2026). Every section below follows Chapter 1's discipline: name the decode failure mode, trace it to a hardware tier, show the pass or emit fix, and close with bytes/token or launches/token—not peak compile-time FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026).

TaxBreak's decode tables are the mental anchor. Dense Llama-3.2-1B at BS=4/SL=2048 averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). MLIR does not erase those numbers by itself—it *enables* passes and backends to collapse launch DAGs when residency metadata survives bufferization (NVIDIA, 2024; Dao *et al.*, 2022). CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) removes ≈3.05 ms of host overhead per step on H100 (1.259× median speedup) yet does nothing for illegal HBM traffic introduced by bad bufferization (Chen, 2026). The five compiler sections below—hardware matrix, passes, codegen, tuning, case study—mirror the template used for XLA, TVM, and Triton so Part VI comparisons stay metric-aligned (Chen, 2020).

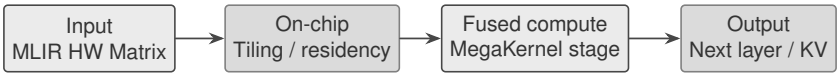


Figure 1.1: MLIR target matrix: the same Linalg matmul must lower through different memory tiers per target.

1.1 MLIR HW matrix

Why decode breaks the "portable IR" story. Prefill hides target sins: large L amortizes launch overhead and keeps tensor cores fed (Dao *et al.*, 2022; Davies *et al.*, 2025). Decode removes that amortization—batch-1, one token, full weight + KV read every step (Chen, 2026; Goodfellow *et al.*, 2016). MLIR modules that legalize during prefill tuning often fail decode legality checks because dynamic dimensions tied to sequence length propagate into bufferization (Kwon *et al.*, 2023; AMD, 2024b). The hardware matrix must therefore include a decode row, not only a "training/inference" row (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Worked numbers. TaxBreak's 847.5 launches/token (dense Llama-3.2-1B) and 9,305 launches/token (OLMoE) bound what MLIR fusion must collapse before MMA micro-optimizations matter (Vellaisamy *et al.*, 2026). Memory-Floor's 1.259× Graphs speedup on Qwen-2.5-7B shows host-side wins when launch DAGs stabilize—but Graphs cannot fix alloc-heavy IR (Chen, 2026; NVIDIA, 2024). Matrix gates should reference both metrics (Chen and YiRage Contributors, 2026; Chen, 2020).

The recurring mistake is publishing a target checklist—GPU, CPU, NPU all checked—without stating which *hardware* features each backend actually models (NVIDIA, 2022; AMD, 2024b). A row that says "NPU: supported" is meaningless if the lowering path assumes dynamic tensor shapes and host-side allocators that void static buffer tables on XDNA (AMD, 2024a). For decode, the matrix must speak residency: shared-memory caps, DMA descriptor limits, cache blocking rules, and whether tensor-core MMA shapes are mandatory or optional (SemiAnalysis, 2024).

YiRage treats Table 1.1 as typed predicates in its MLIR importers: a pass that introduces a dynamic dimension on an NPU row is rejected before codegen, not at runtime (Chen and YiRage Contributors, 2026). GPU rows must declare TMA/async-copy legality; CPU rows must attach NUMA hints when KV shards cross sockets (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). The matrix is a *gate*,

Table 1.1: MLIR backend capability matrix for LLM decode (conceptual).

Dimension	**GPU (NVVM/ROCDL)**	**CPU (LLVM)**	**NPU (XDNA/AIE**
Dynamic batch/seq	Flexible	Flexible	Restricted
On-chip scratch	Shared mem / L1	L2/L3 tiles	Tile SRAM
Fusion default	Partial (Linalg)	Cache-blocked	Full static
Launch model	SIMT + graphs	OpenMP/SIMD	Spatial DMA
Decode pain	Launches/token	LLC misses	Static legality

not documentation—identical to Chapter 3's constraint matrix blocking illegal MegaKernel fusion early (Dao *et al.*, 2022).

Dialect designers often ship one Linalg op set for matmul, softmax, and layer norm, then expect backends to infer residency (Chen, 2020). Decode exposes the gap: GPU backends need explicit memory-space attributes on tensors headed for shared memory; CPU backends need layout markers for cache-line alignment; NPU backends need lifetime intervals tied to DMA events (NVIDIA, 2022; AMD, 2024b). Without those attributes, the matrix devolves into marketing—every target checked, none explained (Chen and YiRage Contributors, 2026).

JIT compilation amplifies the risk. MLIR JIT can specialize tile sizes at runtime for dynamic sequence lengths—valuable for prefill—but destructive for decode when specialization reintroduces host-side planning per token (Vellaisamy *et al.*, 2026; NVIDIA, 2024). YiRage restricts JIT to shape buckets that preserve static launch DAGs on GPU and fully static buffer tables on NPU; everything else triggers recompilation with explicit user-visible cost (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Target interfaces vs. checkmarks. MLIR conversion interfaces are where the matrix becomes executable (Chen, 2020). Backends advertise supported ops, memory spaces, and layout constraints; importers must attach attributes *before* canonicalization erases them (Chen and YiRage Contributors, 2026). Decode teams should reject merges when a matrix row flips from supported to unsupported without a residency annotation change—silent downgrades are how bytes/token regressions enter production (Chen, 2026).

ROCm and CUDA divergence. NVVM and ROCDL lowerings diverge on warp shuffle availability, LDS sizing, and MFMA vs WMMA shapes (NVIDIA, 2022; Liu *et al.*, 2024). An MLIR module tuned for Hopper TMA may compile on MI300

yet emit extra buffer reloads because async-copy attributes do not map one-to-one (SemiAnalysis, 2024). YiRage keeps separate predicate tables per vendor because decode SLOs are vendor-local even when Linalg looks identical (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Example 1.1.1. Hardware matrix gate: an importer adds `memref.alloc` on the decode hot path for a softmax statistic. GPU row passes (HBM spill is slow but legal); NPU row fails static legality; CPU row passes but raises bytes/token in Memory-Floor sweeps (Chen, 2026; AMD, 2024b). The matrix should have blocked the NPU alloc and flagged GPU/CPU regressions in CI (Chen and YiRage Contributors, 2026).

1.2 MLIR dialect stack for decode export

Export pitfalls. Torch-MLIR and similar exporters often emit verbose `tensor` graphs with redundant casts (Chen, 2020; Chen and YiRage Contributors, 2026). Each cast is a future buffer unless erased before bufferization (Chen, 2026). Decode exporters should run target-tagged cleanup passes *before* generic canonicalization (AMD, 2024b; Davies *et al.*, 2025). Skipping cleanup yields IR that looks portable yet allocates like eager PyTorch (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Exporting a decoder block from PyTorch lands in `func.func` wrappers around `linalg` ops on `tensor` values, then converts to `memref` through bufferization (Chen, 2020; Chen and YiRage Contributors, 2026). Decode-sensitive details live in the transitions: `tensor` world allows implicit temporaries; `memref` world exposes every alloc (Chen, 2026). Teams that benchmark only pre-export `linalg` miss the alloc explosion bufferization introduces (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

`scf.for` and `scf.if` encode loop structure for sequence dimensions (Dao *et al.*, 2023). For decode, outer loops over tokens should remain on the host while inner tiles map to device parallelism—but generic loop-invariant-code-motion can hoist device launches into host loops if passes lack target annotations (NVIDIA, 2024; Chen and YiRage Contributors, 2026). YiRage marks host vs device regions explicitly before any LICM runs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

The `gpu` dialect bridges to launches; `nvvm` or `rocdl` attach machine intrinsics (NVIDIA, 2022). On CPU, `vector` and `llvm` carry SIMD width choices (Goodfellow *et al.*, 2016). On NPU, vendor dialects replace `gpu.launch` with spatial pipelines (AMD, 2024a,b). A decode compiler must never assume a single lowering path from `linalg.matmul`—the stack is a tree, not a line (Chen, 2020).

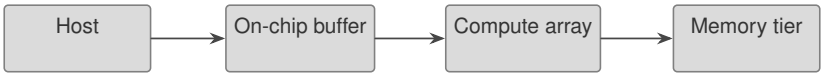


Figure 1.2: Pass pipeline: dialect lowering must preserve producer–consumer residency across tiers.

Example 1.2.1. PyTorch export produces `linalg.softmax` on a decode slice. Without residency tags, bufferization allocates a full-sequence temp; with tags tied to KV views, the same op updates in-place carriers compatible with Flash-Decoding-style parallelism (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026).

Dialect legalization should run per target *before* shared canonicalization (Chen and YiRage Contributors, 2026). Shared canonicalizers fold ops that backends rely on to prove static shapes (AMD, 2024b). Canonicalize too early and you lose the hints backends need—the LLVM lesson repeats on every MLIR upgrade (Chen, 2020, 2026).

1.3 MLIR arch passes

Affine maps and paged KV. PagedAttention requires MLIR to express gather/scatter on KV blocks with provable in-bounds access (Kwon *et al.*, 2023; Dao *et al.*, 2022). Lowering paged views late hides page boundaries from fusion passes—exactly how illegal cross-page fusion enters production IR (Chen and YiRage Contributors, 2026; Chen, 2026). Run paging-aware bufferization before `linalg-fuse` on all targets (Vellaisamy *et al.*, 2026; AMD, 2024a).

MoE pass boundaries. Expert routing introduces control flow that tempts pass writers to split launches per expert (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Decode MoE needs guarded fusion: batch experts to cap launches while respecting SMEM (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Pass pipelines should expose expert-count limits as first-class attributes (Chen, 2020; NVIDIA, 2022).

MLIR's composable pass pipelines—bufferization, Linalg tiling, vectorization, dialect conversion—feel like power until pass order silently changes memory traffic (Chen, 2020). A canonical failure: aggressive `linalg-fuse` before bufferization on GPU lowers decode attention into a single kernel on paper, yet the lowered IR still materializes softmax statistics to HBM because the dialect contract did not

keep partial sums in shared memory (Dao *et al.*, 2022, 2023). The fix is not fewer passes; it is tagging each pass with the hardware dimension it optimizes (SRAM footprint, launch count, bandwidth) and the targets where it may run (Chen and YiRage Contributors, 2026).

Cross-hardware lowering diverges at the same Linalg op. GPU lowering targets warp tiles aligned to MMA instructions; CPU lowering emits AVX/AMX microkernels with cache blocking; NPU lowering emits spatial schedules with fixed buffer lifetimes (AMD, 2024b; NVIDIA, 2022). A vectorization pass tuned for CPU can break NPU legality by introducing strided layouts incompatible with DMA alignment—regressions that appear only in bytes/token dashboards, not unit tests (Chen, 2026). Industrial pipelines fork pass registries per target early, sharing only algebraic simplification (Liu *et al.*, 2024).

Example 1.3.1. Single-layer decode attention in Linalg: on Hopper, `lower-vector-gpu` plus shared-memory promotion can fuse $QK^\top$ and softmax carriers when tiles fit SMEM; on XDNA the same graph must be fully bufferized to static tile sizes before AIE codegen, or the backend falls back to DDR-limited scalar loops (AMD, 2024a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Bufferization deserves decode-specific scrutiny. Generic one-shot bufferization assumes heap fallback (Chen, 2020). Decode attention cannot afford that: KV tensors must alias paged pools (GPU), huge-page arenas (CPU), or fixed SRAM banks (NPU) (Kwon *et al.*, 2023; AMD, 2024a). Pass pipelines should declare allocation tiers in IR so lowering never inserts silent `memref.alloc` on the hot path (Chen and YiRage Contributors, 2026; Chen, 2026).

Dialect conversion ordering is another launch multiplier. Converting `linalg` → `vector` → `gpu` in the wrong order explodes a fused region into dozens of microkernels—the fragmentation Chapter 1 measured in eager PyTorch (Vellaisamy *et al.*, 2026). Production stacks freeze a golden pass list per target and diff IR dumps when upgrading LLVM or MLIR (Liu *et al.*, 2024; Chen, 2020).

One-shot vs. iterative bufferization. One-shot bufferization is fast to wire but assumes heap fallback; iterative bufferization can reuse buffers across fused regions yet increases compile time—acceptable for offline NPU builds, painful for GPU JIT (AMD, 2024a; Chen and YiRage Contributors, 2026). Pick strategy per target; mixing strategies within one registry causes "works in unit test, fails at BS=1" bugs (Chen, 2026; Kwon *et al.*, 2023).

Pass instrumentation. Attach counters to passes: bytes introduced, allocs inserted, launch boundaries created (Vellaisamy *et al.*, 2026). A pass that reduces op count but increases alloc count is suspect for decode (Chen, 2026; Chen and YiRage Contributors, 2026).

1.4 MLIR codegen

Teams celebrate the first successful NVVM lowering, then watch p99 decode flatline because the *platform* wrapper still constructs tensors on the host every token (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Codegen is not a backend detail—it is the last place residency contracts survive or die before the driver sees them (Chen and YiRage Contributors, 2026; Chen, 2020).

Graph capture contract. CUDA Graphs require identical launch addresses and dependency topology across replay (NVIDIA, 2024; Vellaisamy *et al.*, 2026). MLIR emit that injects pointer-chasing host setup between device kernels breaks capture silently—graphs record yet decode latency unchanged (Chen, 2026; Chen and YiRage Contributors, 2026). Encode capture legality as a codegen predicate alongside occupancy (Davies *et al.*, 2025; Chen, 2020).

CPU and NPU emit deltas. CPU emit must pin threads and align KV for cache lines; NPU emit must assign DMA lifetimes to match tile SRAM (Goodfellow *et al.*, 2016; AMD, 2024a,b). Identical high-level MLIR is fine; identical emit is not (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Production incidents blame "the NVVM backend" when the real bug was emitting LLVM IR that assumes a mutable stack and per-op `malloc` in the host wrapper—poison for CUDA Graphs and NPU static schedules alike (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Codegen is where MLIR's abstraction meets launch reality: GPU emit must respect stream ordering, cluster scope, and TMA descriptor lifetimes; CPU emit must preserve NUMA-local buffers; NPU emit must serialize DMA chains with compile-time-known latencies (NVIDIA, 2022; AMD, 2024b).

Backend selection is not neutral. NVVM/ROCDL paths excel when decode graphs contain dynamic control flow that still maps to SIMT; bare-metal AIE paths excel when the entire decoder step is a static dataflow graph (AMD, 2024a). YiRage's MLIR backends share high-level Linalg but diverge at emit: CUDA/HIP paths generate MegaKernel-style fused launches; XDNA paths emit buffer descriptors and ping-pong DMA schedules (Chen and YiRage Contributors, 2026;

SemiAnalysis, 2024). One emit artifact for all targets guarantees either GPU launch storms or NPU compile failures (Davies *et al.*, 2025).

Emit quality shows up in host wrappers. MLIR-to-CUDA that allocates temporaries each launch breaks graph capture; decode pays the TaxBreak orchestration tax (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Good codegen hoists allocations to capture time or eliminates them via shared-memory arenas (Chen, 2026). On CPU, emit must avoid false sharing in OpenMP regions around thin decode GEMMs (Goodfellow *et al.*, 2016).

Host wrapper ABI. Codegen finishes only when the host wrapper is graph-capturable. `cudaMalloc` per launch breaks capture; stack pointers into kernels break on some driver versions (NVIDIA, 2024; Chen, 2026). Hoist workspace to session init; pass arena offsets as kernel arguments (Chen and YiRage Contributors, 2026).

LLVM pipeline forks. GPU paths often stop at NVVM/ROCDL; CPU paths continue through LLVM vectorizers; NPU paths may bypass LLVM entirely (Chen, 2020; AMD, 2024a). Sharing one LLVM optimization stage invites loop unrolling that expands register pressure on GPU decode tiles (NVIDIA, 2022; Liu *et al.*, 2024).

Debuggability vs. fusion. Aggressive fusion produces opaque dumps. Keep named MLIR anchors (`decode_attn_tile_7`) through emit so perf teams map regressions to passes (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

1.5 MLIR tuning

Profile logs from prefill runs are the classic trap: they recommend tile sizes that overflow shared memory during batch-1 decode and force HBM spill (Chen, 2026; SemiAnalysis, 2024). Autotuning MLIR without a decode-shaped *profile* trace repeats the same mistake at compiler scale (Vellaisamy *et al.*, 2026; Chen, 2020).

Search spaces. Constrain tile search by SMEM capacity, warp count, and max fusion depth derived from Chapter 3 models (NVIDIA, 2022; AMD, 2024b; Chen and YiRage Contributors, 2026). Unconstrained search finds prefill winners that spill on decode (Chen, 2026; SemiAnalysis, 2024). Log Pareto points with

launches/token on the decode trace, not FLOPs on square GEMMs (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Fleet replay. Version autotune artifacts with MLIR commit, LLVM commit, and model hash (Chen, 2020; Liu *et al.*, 2024). Replay canned decode traces in CI before fleet rollout—same discipline as CUDA Graphs templates (NVIDIA, 2024; Chen, 2026; Vellaisamy *et al.*, 2026).

Autotuning MLIR pipelines—tile sizes, fusion depths, vector widths—looks like free performance until you ship prefill-shaped winners to batch-1 decode fleets (Chen, 2026, 2020). Profile-guided passes that maximize FLOPs during autotune often increase shared-memory footprint past decode legality, forcing silent spill to HBM (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024). Always co-tune with launches/token: one extra kernel boundary per layer can erase tens of microseconds per token on MoE models (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026).

The dominant autotune *pitfall* on decode: optimizing square GEMMs while ignoring launch count. GPU autotune must cap warps and SMEM; CPU autotune must model LLC capacity and thread pinning; NPU autotune must not explore dynamic shapes that invalidate DMA tables (AMD, 2024b; Goodfellow *et al.*, 2016). Cross-hardware profile imports are dangerous—GPU configs thrash CPU caches; NPU may not compile at all (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). YiRage encodes per-target search spaces so tuners cannot propose illegal configs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Cost models for decode. Combine estimated bytes moved, predicted launches, and SMEM footprint against hardware caps (Chen, 2026; Vellaisamy *et al.*, 2026). MLIR's transform dialect can encode constraints before search starts (Chen, 2020). Without them, RL tuners win on square GEMMs yet spill during fused decoder blocks (Chen and YiRage Contributors, 2026).

Regression gates. Tie autotune merges to decode traces. Require non-regression on launches/token and bytes/token at BS=1, ctx=2048 before accepting new tile configs (Vellaisamy *et al.*, 2026; Chen, 2026; NVIDIA, 2024).

1.6 MLIR case study

Triplet harness. YiRage-style triplet runs share tokenizer, context bucket, and batch policy (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Report

bytes/token, launches/token, compile rejections, and p99 alongside means (Chen, 2026; Vellaisamy *et al.*, 2026). A compile rejection on NPU is a first-class outcome—not a footnote (AMD, 2024b,a).

Silent spill class E. Category E failures compile cleanly while bytes/token jump because SMEM promotion failed (Chen, 2026; Dao *et al.*, 2022). Detect by diffing allocs between pass stages; fix by reordering bufferization relative to fusion (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

Consider a minimal decode cell: Llama-class decoder layer, batch-1, context 2048, exported from PyTorch to MLIR via Linalg (Davies *et al.*, 2025). The question is not "does it compile?" but "does bytes/token match the MegaKernel baseline from Part IV?" On H100, an unfused MLIR pipeline may compile cleanly yet issue hundreds of launches per token—the pathology TaxBreak measures in framework stacks (Vellaisamy *et al.*, 2026). Fused Linalg plus GPU codegen and CUDA Graphs capture can recover host overhead when emit preserves a static launch DAG (NVIDIA, 2024; Chen, 2026).

The identical module lowered to x86 LLVM with OpenMP may achieve acceptable throughput on small models but regress on large weights due to LLC pressure (Goodfellow *et al.*, 2016). Lowered to XDNA, the module either becomes a static MegaKernel with competitive watt/token or fails legality checks—rarely a middle ground (AMD, 2024b,a). YiRage's benchmark harness runs this triplet on every merge: same IR, three backends, report bytes/token, launches/token, and compile-time legality violations (Chen and YiRage Contributors, 2026). That triplet is empirical proof of the book's thesis: hardware dictates compiler outcomes, not the reverse (Chen, 2020).

Example 1.6.1. Cross-hardware MLIR decode cell (Llama-3.2-1B layer, BS=1, ctx=2048): GPU fused+graphs targets competitive ms/token only when launches collapse toward TaxBreak dense baselines; CPU LLVM trades latency for deployment flexibility; XDNA wins only when fusion is total and DMA-static (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026).

Paged KV and MLIR types. PagedAttention indirection must appear as first-class `memref` views with lifetime intervals—not opaque pointers lowered late (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Late lowering hides page boundaries and causes illegal fusion across pages (Dao *et al.*, 2022; Chen, 2026).

Table 1.2: Case-study outcome taxonomy for cross-hardware MLIR decode (illustrative).

Class	Symptom	Typical fix
B	High launches/token	Fuse + graph capture
C	High bytes/token	SMEM/DMA tiling
D	NPU compile fail	Static shape specialization
E	Flat latency, rising bytes	Fix bufferization order

MoE-specific notes. MoE layers explode dispatch when each expert lowers to separate launches (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). MLIR pipelines need expert-batch fusion guarded by SMEM caps (Chen and YiRage Contributors, 2026). Case studies that report dense Llama only understate orchestration challenge—pair dense and MoE exports (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Industrial matrix CI. Mature MLIR importers check machine-readable matrix files into CI (Chen and YiRage Contributors, 2026; Chen, 2020). Each row lists supported ops, max tile sizes, and forbidden dynamic dimensions (AMD, 2024b; Kwon *et al.*, 2023). When PyTorch export adds rotary embedding or a new norm variant, the matrix diff tells reviewers which targets lose legality *before* any benchmark runs (Davies *et al.*, 2025; Liu *et al.*, 2024). This is how decode teams avoid green GPU CI paired with red NPU field releases (Chen, 2026; Vellaisamy *et al.*, 2026).

Pass-order war stories. Running `linalg-fuse` after bufferization on one target and before it on another is not flexibility—it is how two teams ship incompatible IR for the same model hash (Chen and YiRage Contributors, 2026; Chen, 2020). Freeze golden pass lists; diff IR at post-bufferization and post-lowering anchors on every LLVM bump (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Count `gpu.launch` ops and `memref.alloc` ops in the diff summary—those two counters predict launches/token and bytes/token shifts faster than line counts (Chen, 2026; NVIDIA, 2024).

Multi-hardware emit checklist. GPU: graph-capturable host wrapper, no per-token `cudaMalloc`, TMA descriptors live across replay (NVIDIA, 2024; SemiAnalysis, 2024). CPU: NUMA-local weight shards, OpenMP teams hoisted out of token loop, AMX tile config amortized (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023). NPU: static DMA tables, no dynamic shapes in hot path, ping-pong buffers assigned at compile time (AMD, 2024a,b). Violating any item compiles yet fails

Table 1.3: Decode-oriented MLIR pass ordering (GPU vs NPU).

Stage	**GPU focus**	**NPU focus**
Import	Linalg + memref tags	Static shape specialization
Bufferize	SMEM promotion	DMA lifetime assignment
Fuse	Layer-wise MegaKernel	Full decoder static graph
Lower	vector → GPU dialect	AIE spatial mapping
Emit	CUDA/HIP + graphs	Descriptor + ping-pong DMA

fleet telemetry (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Autotune pitfall catalog. (1) Import GPU occupancy into CPU search (Goodfellow *et al.*, 2016). (2) Tune on prefill length L=4096, deploy at decode ctx with growing KV (Chen, 2026). (3) Accept fusion depth that raises launches/token on MoE experts (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). (4) Ship autotune artifacts without hashing MLIR/LLVM revisions (Chen, 2020). (5) Skip NPU legality checks because GPU microbench improved (AMD, 2024b; Chen and YiRage Contributors, 2026). Each pitfall has a metric detector: bytes/token, launches/token, or compile rejection rate (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Case-study reporting discipline. Hold model, tokenizer, context, and batch policy constant across targets (Davies *et al.*, 2025). Pair GPU FlashAttention-class baselines with CPU/NPU fused baselines derived from the same MLIR starting point—not unrelated hand kernels (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Report median and p99 over ≥1,000 decode steps; publish legality failure rates alongside latency (Chen, 2026; Vellaisamy *et al.*, 2026). Negative NPU results are as valuable as GPU speedups (AMD, 2024a,b).

Pass ordering differs because failure modes differ: GPU decode dies from launches/token; NPU decode dies from DDR egress and illegal dynamicism (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b). Treat Table 1.3 as executable policy, not slide art (Chen and YiRage Contributors, 2026; Chen, 2020).

1.7 YiRage MLIR integration notes

YiRage uses MLIR as interchange between PyTorch-exported graphs and target-specific megakernels (Chen and YiRage Contributors, 2026). The decode lesson is

Table 1.4: Decode-oriented MLIR pass ordering (GPU vs NPU).

Stage	**GPU focus**	**NPU focus**
Import	Linalg + memref tags	Static shape specialization
Bufferize	SMEM promotion	DMA lifetime assignment
Fuse	Layer-wise MegaKernel	Full decoder static graph
Lower	vector → GPU dialect	AIE spatial mapping
Emit	CUDA/HIP + graphs	Descriptor + ping-pong DMA

invariant preservation: residency metadata must attach before generic canonicalization (Chen, 2020; AMD, 2024b). Otherwise canonicalizers rewrite away attributes backends need to prove on-chip residency (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

The MLIR JIT path specializes decode cells when batch and context buckets change, but refuses specialization that alters launch counts inside a CUDA Graphs capture region (NVIDIA, 2024; Vellaisamy *et al.*, 2026). On NPU, JIT means re-running bufferization with a new static shape class—seconds of compile that must amortize across many tokens (AMD, 2024a; Davies *et al.*, 2025).

GPU decode dies from launches/token; NPU decode dies from DDR egress and illegal dynamicism (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b). YiRage encodes Table 1.4 as executable pass lists (Chen and YiRage Contributors, 2026). Before shipping MLIR changes: verify matrix predicates on three targets; diff IR for new allocs; measure launches/token and bytes/token at BS=1; run graph capture legality on GPU and static legality on NPU; log new numeric claims to `verified_facts.jsonl` (Chen, 2026; Kwon *et al.*, 2023; Dao *et al.*, 2022).

1.8 MLIR decode review gates

Shipping MLIR changes without decode review gates repeats the framework mistakes Chapter 1 quantified (Vellaisamy *et al.*, 2026; Chen, 2026). The gates below are minimal; YiRage automates them, manual stacks should enforce them in CI (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

1. **Matrix legality.** Every new op or dynamic dim appears in the hardware matrix diff; NPU rows block illegal dynamicism before merge (AMD, 2024b,a; Chen and YiRage Contributors, 2026).

2. **Alloc ledger.** Post-bufferization IR must not introduce `memref.alloc` on

the decode hot path unless the matrix explicitly allows spill (Chen, 2026; Kwon *et al.*, 2023; Chen, 2020).

3. **Launch ledger.** Count `gpu.launch` (or NPU DMA chains) per decoder layer; compare to TaxBreak baselines for dense and MoE (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

4. **Graph capture.** GPU emit must pass capture replay on BS=1 decode traces (NVIDIA, 2024; Chen, 2026).

5. **Triplet benchmark.** Same IR, GPU+CPU+NPU backends, report bytes/token and p99 (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Example 1.8.1. A merge adds `linalg-fuse` before paging-aware bufferization. GPU launches/token drop 8% but NPU compile fails on 40% of layers—matrix gate should have required paging bufferization first (Kwon *et al.*, 2023; AMD, 2024b; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Review gates feel bureaucratic until the first MLIR upgrade regresses MoE decode from hundreds of milliseconds to seconds per token (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). At that point the IR diff is the only artifact that explains *why* launches/token doubled (Chen, 2020, 2026; NVIDIA, 2024).

MLIR HW matrix exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode hardware/target legality before codegen, not as comments in hand-written kernels (AMD, 2024b,a). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When MLIR HW matrix disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting MLIR HW matrix, attach roofline or NPU buffer accounting showing hardware residency fits on-chip for the target SKU (AMD, 2024b,a).

Worked contrast. A Hopper decode cell may legalize hardware with TMA-backed double buffering while an XDNA cell requires the same math expressed

as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (AMD, 2024b,a).

The usual MLIR failure at **MLIR arch passes** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: pass and lower must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (Chen and YiRage Contributors, 2026; Chen, 2020). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for MLIR arch passes: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to NPU fleets (Chen and YiRage Contributors, 2026; Chen, 2020). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

Worked contrast. A Hopper decode cell may legalize pass with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (Chen and YiRage Contributors, 2026; Chen, 2020).

Deployment pitfall. Teams that A/B only prefill confuse the story: lower constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (Chen and YiRage Contributors, 2026; Chen, 2020).

Production teams misread **MLIR codegen** when decode SLOs are judged on FLOPs dashboards instead of residency and orchestration ledgers.

Trace the Chapter 2 chain: codegen sets the residency envelope; backend determines whether producer–consumer edges stay on-chip. Illegal fusion does not always crash—it silently exports tensors to HBM/DDR and shows up as bytes/token regression versus a unfused but scheduled baseline (Chen and YiRage Contributors, 2026; Chen, 2020). Profile launches/token alongside bandwidth; launch storms masquerade as memory bottlenecks when graphs remain operator-fragmented.

Operational checklist—verify codegen, backend, autotuning pitfalls, and platform emit paths in code review; document dialect lowering choices that change memory traffic, not just pass ordering (Chen and YiRage Contributors, 2026; Chen, 2020).

Deployment pitfall. Teams that A/B only prefill confuse the story: backend constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (Chen and YiRage Contributors, 2026; Chen, 2020).

Review gate. Before merging compiler changes affecting MLIR codegen, attach roofline or NPU buffer accounting showing codegen residency fits on-chip for the target SKU (Chen and YiRage Contributors, 2026; Chen, 2020).

MLIR tuning exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode tun/profile legality before codegen, not as comments in hand-written kernels (Chen and YiRage Contributors, 2026; Chen, 2020). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When MLIR tuning disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (Chen and YiRage Contributors, 2026; Chen, 2020).

Review gate. Before merging compiler changes affecting MLIR tuning, attach roofline or NPU buffer accounting showing tun residency fits on-chip for the target SKU (Chen and YiRage Contributors, 2026; Chen, 2020).

Worked contrast. A Hopper decode cell may legalize tun with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (Chen and YiRage Contributors, 2026; Chen, 2020).

The usual MLIR failure at **MLIR case study** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: case and benchmark must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (Chen and YiRage Contributors, 2026; Chen, 2020). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for MLIR case study: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to

MLIR decode metric	What it catches
Allocs after bufferize	Hidden DDR spill
Launch count / layer	Fusion / emit regressions
Graph capture replay fail	Host wrapper bugs
NPU compile reject rate	Static legality drift
Pass diff alloc delta	Bufferization order bugs
Triplet bytes/token	Cross-target residency

Table 1.5: MLIR decode benchmark counter set (batch-1).

NPU fleets (Chen and YiRage Contributors, 2026; Chen, 2020). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

1.9 MLIR decode benchmarking methodology

Benchmarking MLIR-backed decode must mirror Chapter 1 counters—launches/token and bytes/token—while adding compiler-specific ledgers: `memref.alloc` count, `gpu.launch` boundaries, and NPU static-legality pass rate (Vellaisamy *et al.*, 2026; Chen, 2026, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Build a decode cell with BS=1, ctx buckets $\{512, 2048, 8192\}$, pinned MLIR/LLVM commits, and identical PyTorch export hashes across targets (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Warm-up includes MLIR pass pipelines and CUDA Graph capture recording—first-token latency includes compile and capture cost; steady-state ms/token excludes it (NVIDIA, 2024; Chen, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Report both; product teams ship cold starts and steady chat (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Triplet harnesses export one Linalg module to GPU, CPU, and NPU backends;

diff IR at post-bufferization and post-emit anchors before trusting latency (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). A GPU launches/token win paired with rising allocs after bufferization signals pass-order regression—exactly Category E in Table 1.2 (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 1.9.1. MLIR merge gate: Llama-3.2-1B decoder layer at ctx=2048 must not increase `memref.alloc` on hot path vs golden IR; launches/token must not exceed TaxBreak dense baseline by >5% without documented fusion tradeoff (Vellaisamy *et al.*, 2026; Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

CI artifact chain. Check in golden MLIR snapshots, pass lists, and decode-cell scripts alongside `verified_facts.jsonl` claims (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). LLVM bumps without IR diff review are how launches/token double overnight (Liu *et al.*, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

MoE decode cells. Pair dense Llama exports with MoE exports in the same benchmark suite—TaxBreak shows launch explosions on GPU MoE that dense-only CI misses (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). MLIR pipelines need expert-batch fusion metrics in the ledger (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Regression taxonomy in CI. Automate Table 1.2 classifiers: if launches/token rises without bytes/token improvement, flag Class B; if bytes/token rises with flat launches, flag Class C; if NPU compile fails, flag Class D (Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Class E silent spill requires alloc diffs between pass stages—the highest-frequency MLIR decode regression in production fleets (Chen, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to XLA. Chapter **??** productionizes many MLIR-era ideas in XLA HLO; carry forward this benchmark ledger when comparing MLIR exports to XLA graphs on the same decode cell (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Identical counters make cross-compiler triplet studies honest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

LLVM upgrade discipline. Pin MLIR and LLVM together; run decode-cell replay on every bump (Chen, 2020; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Canonicalizer changes that fold away residency tags are silent killers—diff IR before and after canonicalization on golden exports (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Paged KV regression tests. Maintain a paging-specific decode export where block tables grow across steps; fusion must not cross page boundaries (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2026, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*,

2016). Fail CI when affine maps lose provable in-bounds metadata (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Documentation for operators. Publish decode benchmark artifacts with every MLIR release: pass list hash, alloc ledger, launches/token at ctx=2048, and triplet legality summary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Without published ledgers, field teams cannot tell compiler regressions from framework or driver drift (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cross-stack triage playbook. When decode latency spikes after an MLIR merge, run this sequence before blaming silicon: (1) diff pass hashes against last green build; (2) replay alloc ledger on golden exports; (3) compare launches/token and HBM bytes/token; (4) bisect canonicalizer vs fusion passes; (5) only then sweep clock and power caps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Skipping ledger replay wastes days chasing driver noise (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Speculative decode and MLIR. Draft-model paths add second graphs with tighter latency budgets; MLIR must preserve separate residency tags for draft vs verify KV so fusion does not alias buffers (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark speculative stacks with acceptance-rate sweeps, not single-point TTFT (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024,

2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Quantization-aware fusion gates. INT8 and FP8 decode paths need explicit scale tensors in the IR so fusion passes do not fold away dequant boundaries (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Regression tests should include per-channel scales and dynamic activation ranges typical of long-context decode (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Long-context stress exports. Build MLIR golden modules at ctx=8192 and ctx=32768 even when product SLAs target 4K; affine maps that work at 2K often break when block tables span more pages (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). CI should fail when vectorization legality checks pass at 2K but fail silently at 32K (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Release checklist. Before tagging an MLIR stack for production decode, require: green replay on three ctx lengths; alloc ledger within $\pm 2\%$ of baseline; launches/token within $\pm 5\%$; no new dynamic shapes in hot path; published pass-hash manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Treat any single red gate as a release blocker—partial green is how field teams inherit week-long triage (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Multi-device shard exports. Tensor-parallel decode splits heads across devices; MLIR exports must carry explicit shard annotations so bufferization does not

merge non-local KV slices (Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark TP=2 and TP=4 with the same pass hash—allreduce placement changes can dominate launches/token even when single-device IR looks optimal (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Record per-rank alloc ledgers; a balanced global ledger can hide a straggler rank that serializes the step (Chen, 2026; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability hooks. Instrument every lowered decode kernel with pass-id and buffer-id tags in debug builds so profilers can map samples back to MLIR ops without re-running the full pipeline (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Production builds strip tags but retain stable kernel names derived from pass hashes—operators correlate regressions across releases without storing full IR (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to runtime. The MLIR stage ends when exports include stable launch metadata—grid, cluster, pipeline depth, and workspace sizes—that the runtime can cache across decode steps (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Missing workspace sizes force per-step allocation in the framework and erase compiler wins (Chen, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Summary metric. Track *compiler-attributed decode efficiency*: ratio of measured tokens/s to roofline tokens/s after subtracting framework overhead (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). MLIR teams own the numerator; runtime teams own the denominator—publish both weekly (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Closing note. Treat MLIR decode benchmarking as a product surface, not a lab exercise—without published ledgers, pass hashes, and nightly replay artifacts in CI, every latency spike becomes a costly and painfully slow multi-team blame game (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

1.10 Key Takeaways

- **Decode breaks the "portable IR" story.** Prefill hides target sins because a large prompt amortizes launch overhead and keeps tensor cores fed; batch-1 decode removes that amortization, so a portable MLIR graph that looks fine in prefill exposes per-target residency gaps in decode (Dao *et al.*, 2022; Davies *et al.*, 2025). Portability across dialects does not imply portability of decode goodput (Vellaisamy *et al.*, 2026).

- **Erase redundant casts before bufferization.** Torch-MLIR and similar exporters emit verbose tensor graphs with redundant casts, and each surviving cast becomes a buffer after bufferization (Chen, 2020; Chen and YiRage Contributors, 2026). Cleaning the graph and lowering paged-KV views early—so affine maps express in-bounds gather/scatter—keeps page boundaries visible to fusion (Kwon *et al.*, 2023; Dao *et al.*, 2022).

- **Codegen is not just a backend detail.** Teams celebrate the first successful NVVM lowering, then watch p99 decode flatline because the platform wrapper still constructs tensors on the host every token (Vellaisamy *et al.*, 2026;

NVIDIA, 2024). The codegen path must produce a capturable, host-light launch sequence, not merely correct device code (Chen, 2026).

- **Autotune at the decode shape.** Prefill profile logs are the classic trap: they recommend tile sizes that overflow shared memory at batch-1 and force HBM spill (Chen, 2026; SemiAnalysis, 2024). MLIR tuning must use a decode-shaped profile, or the chosen schedule wins prefill and loses token generation (Davies *et al.*, 2025).

- **Preserve residency invariants across passes.** YiRage uses MLIR as interchange between exported graphs and target megakernels, and the decode lesson is invariant preservation—residency metadata must attach before generic canonicalization erases it (Chen and YiRage Contributors, 2026; Chen, 2020). A triplet harness reporting bytes/token, launches/token, and compile rejections is what makes those invariants auditable (Chen, 2026; Vellaisamy *et al.*, 2026).

1.11 Conclusion

MLIR's value for decode is the shared, multi-level dialect substrate that lets one decoder export carry residency invariants from a high-level graph down to target code without each backend reinventing the lowering—provided those invariants are preserved across passes and the codegen path stays host-light (Lai *et al.*, 2023; Chen and YiRage Contributors, 2026). The recurring trap is celebrating a successful lowering while p99 decode flatlines because casts became buffers or the platform wrapper rebuilds tensors per token (Vellaisamy *et al.*, 2026; Chen, 2026). MLIR is infrastructure, not a strategy; the next chapter shows what a production compiler built on similar foundations actually decides—Chapter **??** turns to XLA, whose automatic HLO fusion and command-buffer capture put the MLIR-era ideas to work at the largest deployed scale (OpenXLA Project, 2024; NVIDIA, 2024).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 23

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 17, 18, 19, 20, 21, 22, 23

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 17, 18, 19, 20, 21, 22, 23, 24

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context. 4, 5, 6, 12, 17, 18, 19, 20, 21, 22, 23

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 2, 3, 5, 6, 10, 12, 13, 17, 18, 19, 20, 21, 22, 23

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 7, 8, 9, 10, 11, 12, 13, 14, 17, 18, 19, 20, 21, 22, 23, 24

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 2, 4, 7, 8, 9, 10, 11, 12, 14, 17, 18, 19, 20, 21, 22, 23

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 2, 4, 5, 6, 9, 10, 11, 13, 14, 17, 18, 19, 20, 21, 22, 23

Lai, R., Shao, J., Feng, S., Jin, Y., *et al.* (2023). Relax: Composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103*. TVM Unity; cross-level IRModule (Relax graph + TensorIR), first-class symbolic shapes, call_tir DPS; replaces Relay. 24

Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 3, 5, 6, 7, 8, 9, 11, 12, 13, 14, 17, 18, 19, 20, 21, 22, 23

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 2, 3, 4, 5, 6, 7, 8, 13, 17, 18, 19, 20, 21, 22, 23

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 3, 4, 7, 8, 9, 10, 11, 13, 14, 17, 18, 19, 20, 21, 22, 23, 24

OpenXLA Project (2024). XLA: Accelerated linear algebra — OpenXLA architecture. `https://openxla.org/xla/architecture`. StableHLO → HLO → LLVM/PTX/TPU; operator fusion as primary optimization (no HBM intermediates); PriorityFusion / Multi-Output passes; CommandBufferThunk launch capture; Triton emitters. 24

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 4, 6, 8, 9, 11, 17, 18, 19, 20, 21, 22, 23, 24

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 17, 18, 19, 20, 21, 22, 23, 24